

Agentic Bootcamp: Qdrant on GCP

Installation and N8N-VectorDB Integration Guide

Contents

1	Overview	2
2	Creating a Qdrant SaaS Account on GCP (Free Tier)	2
3	Creating an Example Vector Store (Collection) via UI	2
4	Automating Qdrant with n8n	5
4.1	Configuring the Qdrant Node Credentials	5
4.2	Managing Collections via n8n Operations	5
4.2.1	1. Create Collection	5
4.2.2	2. List Collections	5
4.2.3	3. Update Collection	6
5	Uploading Data (Vectors) from JSON and CSV	6
5.1	Ingesting Data from a CSV File	6
5.2	Ingesting Data from a JSON File	7
6	Viewing Vector Graphs and HNSW Index Insights	8
6.1	Inspecting HNSW Metrics via Qdrant UI	8
6.2	Extracting HNSW Graph Telemetry via Python API	8
6.3	Visualizing Vector Embeddings (dimensionality reduction using UMAP/t-SNE) .	8
7	Data Update and Retrieval Strategies (Python Code Snippets)	9
7.1	Strategy 1: Vector & Payload Upsert (Insert or Update)	9
7.2	Strategy 2: Similarity Top-K Vector Search	10
7.3	Strategy 3: Filtered Top-K Similarity Search (Hybrid Filtering)	11
8	Summary	11

1 Overview

Qdrant is an open-source, high-performance vector search engine designed for vector embeddings and payload filtering. This guide outlines the end-to-end setup of a Qdrant SaaS (Cloud) instance on Google Cloud Platform (GCP) using the Free Tier, followed by hands-on collection management, data ingestion, visual debugging, and Python-based data retrieval strategies.

2 Creating a Qdrant SaaS Account on GCP (Free Tier)

Follow these detailed steps to provision a free Qdrant cloud cluster hosted on Google Cloud Infrastructure:

1. **Navigate to Qdrant Cloud Portal:** Open your web browser and navigate to <https://cloud.qdrant.io>.
 2. **Sign Up / Registration:**
 - Click on **Sign Up**.
 - Register using your corporate email, Google SSO, or GitHub account.
 - Verify your email address if prompted.
 3. **Create a New Cluster:**
 - Once inside the Qdrant Cloud Dashboard, click the **Create Cluster** button.
 - In the **Cloud Provider** section, select **Google Cloud Platform (GCP)**.
 - Select your preferred GCP region (e.g., `us-central1` or `eu-west3`).
 4. **Select the Free Tier Instance:**
 - Under the cluster configuration tier, select **Free Tier** (typically provides 1 GB RAM, 0.5 vCPU, and up to 4M 384-dim vectors).
 - Enter a name for your cluster (e.g., `gcp-qdrant-dev-cluster`).
 - Click **Create Cluster**.
 5. **Generate API Keys and Store Cluster Credentials:**
 - Once the cluster state changes to **Healthy / Active**, navigate to the **API Keys** section in the left sidebar.
 - Click **Create API Key**, set permissions to Read/Write, and copy the generated key.
 - Copy your **Cluster Endpoint URL**
e.g., `https://<cluster-id>.<region>.gcp.cloud.qdrant.io:6333`.
-

3 Creating an Example Vector Store (Collection) via UI

Qdrant provides a built-in Cloud Console and Dashboard UI to manage vector collections visually.

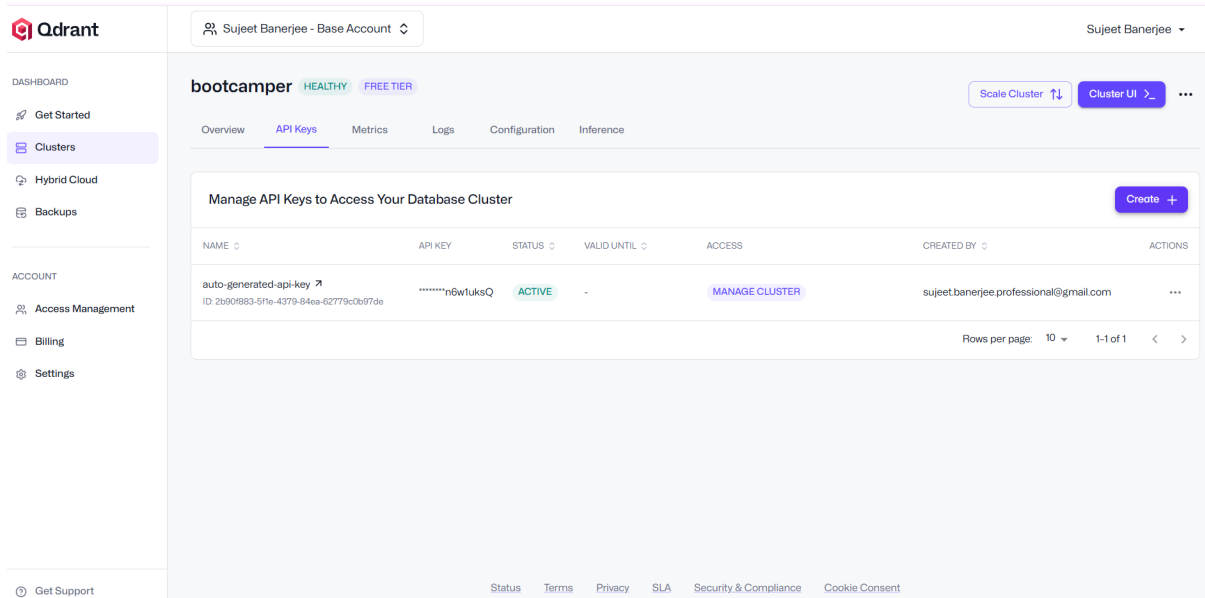


Figure 1: Qdrant Create API Key

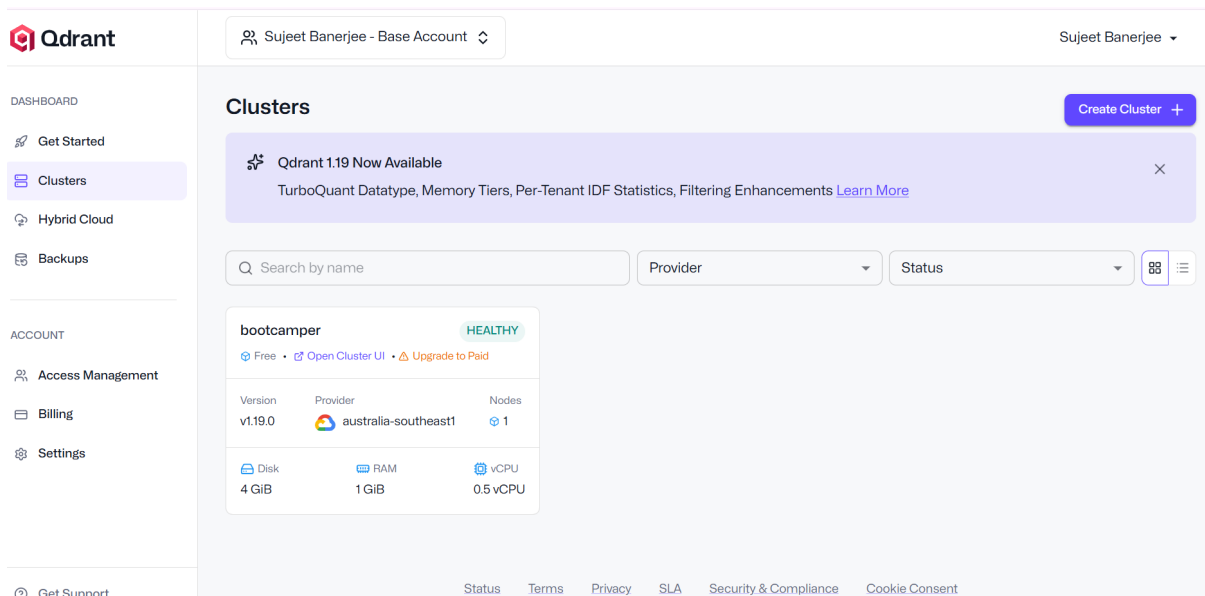


Figure 2: Qdrant Dashboard

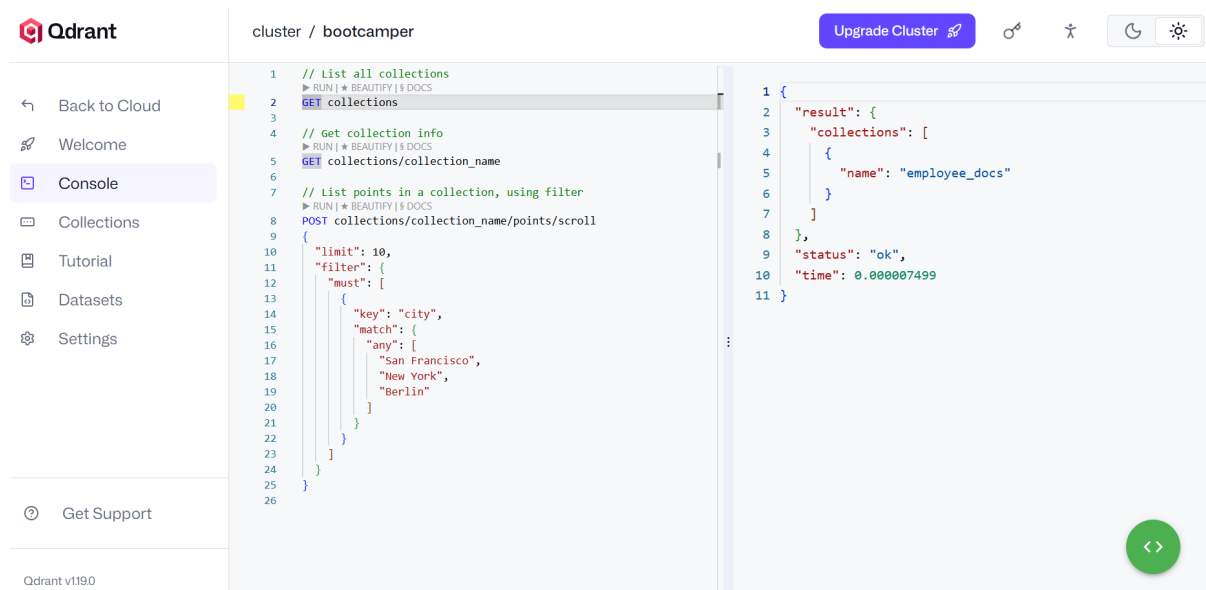


Figure 3: Qdrant Interactive Console

1. **Access the Qdrant Web UI Dashboard:** In the Qdrant Cloud console, click on your active cluster and select **Console** or click **Open Dashboard**. Alternatively, navigate to <https://<cluster-endpoint>:6333/dashboard>.
2. **Open the Interactive Console:** On the left navigation pane, select **Console** (or REST API Client inside Dashboard).
3. **Create Collection via UI / REST Console:** In the interactive HTTP request window, execute a PUT request to create a collection named `ecommerce_products`:

```
1 PUT /collections/ecommerce_products
2 {
3   "vectors": {
4     "size": 384,
5     "distance": "Cosine"
6   },
7   "hnsw_config": {
8     "m": 16,
9     "ef_construct": 100
10  }
11 }
```

Listing 1: REST Payload to Create Collection

Refer to the Figure 3.

4. Verify Collection Parameters:

- Navigate to the **Collections** tab in the Dashboard.
- Select `ecommerce_products`.
- Verify that the **Vector Dimension** is set to 384, the **Distance Metric** is set to *Cosine*, and status indicates **Green / Ready**.

4 Automating Qdrant with n8n

n8n is a powerful workflow automation tool that provides a native Qdrant node, allowing you to visually manage your vector database and integrate it into low-code data pipelines.

4.1 Configuring the Qdrant Node Credentials

Before performing operations, you must connect n8n to your GCP-hosted Qdrant Cloud instance:

1. Add a new **Qdrant** node to your n8n workflow canvas.
2. Open the node settings and under **Credentials**, select **Create New Credential** → **Qdrant API**.
3. In the credentials configuration window:
 - **URL**: Enter your GCP cluster endpoint (e.g., `https://<cluster-id>.us-central1.gcp.cloud.qdrant.io`).
 - **API Key**: Paste your Qdrant Cloud API key.
4. Click **Save** to authenticate.

4.2 Managing Collections via n8n Operations

Within the Qdrant node settings, set the **Resource** dropdown to **Collection**. You can then select specific operations to manage your vector stores.

4.2.1 1. Create Collection

Use this operation to instantiate a new collection to hold your vectors.

- **Operation**: Select **Create**.
- **Collection Name**: Enter the desired identifier (e.g., `customer_support_docs`).
- **Vector Size**: Input the exact dimensionality of your embedding model (e.g., 384 or 1536).
- **Distance**: Select your preferred similarity metric (**Cosine**, **Euclid**, or **Dot**).
- **Additional Fields**: Optionally add fields to configure **HNSW Config** (e.g., setting `m` or `ef_construct`) directly at creation.

4.2.2 2. List Collections

Use this operation to fetch all active collections in your Qdrant cluster.

- **Operation**: Select **Get Many** (or **List**, depending on the n8n version).
- **Output**: Executing this node will return a JSON array in the output pane, detailing all collection names and their deployment status on your GCP node, which can be passed to subsequent nodes for iteration or conditional logic.

4.2.3 3. Update Collection

Use this operation to dynamically modify the indexing and optimization parameters of an existing collection.

- **Operation:** Select Update.
- **Collection Name:** Enter the name of the collection you wish to modify.
- **Update Parameters:** Click **Add Field** to append parameters to update:
 - **Optimizers Config:** Update fields like `deleted_threshold` or `vacuum_min_vector_number` to manage background disk operations and memory.
 - **HNSW Config:** Update graph indexing settings to favor search speed over accuracy (or vice versa) without re-uploading the data.

—

5 Uploading Data (Vectors) from JSON and CSV

Data ingestion is performed using the Python SDK (`qdrant-client`). Install required libraries first:

```
1 pip install qdrant-client pandas sentence-transformers
```

5.1 Ingesting Data from a CSV File

Assume a CSV file `products.csv` containing columns: `id`, `name`, `category`, `price`, and `description`.

```
1 import pandas as pd
2 from qdrant_client import QdrantClient
3 from qdrant_client.models import PointStruct, Distance, VectorParams
4 from sentence_transformers import SentenceTransformer
5
6 # Initialize Client
7 client = QdrantClient(
8     url="https://<your-cluster-id>.gcp.cloud.qdrant.io:6333",
9     api_key="<YOUR_API_KEY>"
10 )
11
12 COLLECTION_NAME = "ecommerce_products"
13
14 # Load Embedding Model
15 encoder = SentenceTransformer('all-MiniLM-L6-v2') # Output dim: 384
16
17 # Read CSV Data
18 df = pd.read_csv("products.csv")
19
20 # Generate Embeddings and Prepare Points
21 points = []
22 for idx, row in df.iterrows():
23     text_to_embed = f"{row['name']} - {row['description']}"
24     vector = encoder.encode(text_to_embed).tolist()
25
26     point = PointStruct(
27         id=int(row['id']),
28         vector=vector,
29         payload={
```

```

30         "name": row['name'],
31         "category": row['category'],
32         "price": float(row['price'])
33     }
34 )
35 points.append(point)
36
37 # Upload Batch to Qdrant
38 client.upsert(
39     collection_name=COLLECTION_NAME,
40     points=points
41 )
42 print(f"Successfully uploaded {len(points)} records from CSV.")

```

Listing 2: Upload CSV Vectors to Qdrant

5.2 Ingesting Data from a JSON File

Assume a JSON file `products.json` formatted as an array of JSON objects.

```

1 import json
2 from qdrant_client import QdrantClient
3 from qdrant_client.models import PointStruct
4 from sentence_transformers import SentenceTransformer
5
6 client = QdrantClient(
7     url="https://<your-cluster-id>.gcp.cloud.qdrant.io:6333",
8     api_key="<YOUR_API_KEY>"
9 )
10
11 encoder = SentenceTransformer('all-MiniLM-L6-v2')
12
13 with open("products.json", "r") as f:
14     data = json.load(f)
15
16 points = []
17 for item in data:
18     text = item['description']
19     vector = encoder.encode(text).tolist()
20
21     points.append(PointStruct(
22         id=item['id'],
23         vector=vector,
24         payload={
25             "title": item['title'],
26             "tags": item.get('tags', []),
27             "in_stock": item.get('in_stock', True)
28         }
29     ))
30
31 # Batch Upload
32 client.upload_points(
33     collection_name="ecommerce_products",
34     points=points,
35     batch_size=64
36 )
37 print("JSON data successfully uploaded.")

```

Listing 3: Upload JSON Vectors to Qdrant

6 Viewing Vector Graphs and HNSW Index Insights

Hierarchical Navigable Small World (HNSW) is the underlying graph indexing algorithm used by Qdrant for approximate nearest neighbor (ANN) search.

6.1 Inspecting HNSW Metrics via Qdrant UI

1. Open the **Qdrant Cloud Dashboard** and select your collection (`ecommerce_products`).
2. Navigate to the **Info / Stats** panel to view real-time vector statistics:
 - **Status:** green (indicates all segments are indexed).
 - **Indexed Vectors Count:** Number of vectors built into the HNSW graph.
 - **Segments Count:** View RAM vs. Disk segments.
3. Under **Collection Configuration**, inspect HNSW parameters:
 - *m*: Number of edges per node in the construction graph (e.g., 16).
 - *ef_construct*: Size of dynamic candidate list during graph construction (e.g., 100).

6.2 Extracting HNSW Graph Telemetry via Python API

You can query collection cluster telemetry programmatically:

```
1 from qdrant_client import QdrantClient
2
3 client = QdrantClient(url="https://<cluster-endpoint>:6333", api_key="<
    API_KEY>")
4
5 info = client.get_collection(collection_name="ecommerce_products")
6 print("Status:", info.status)
7 print("Vectors Count:", info.vectors_count)
8 print("Indexed Vectors Count:", info.indexed_vectors_count)
9 print("HNSW Config:", info.config.hnsw_config)
```

Listing 4: Fetch HNSW and Collection Metrics

6.3 Visualizing Vector Embeddings (dimensionality reduction using UMAP/t-SNE)

To visually inspect vector distances as a graph in 2D/3D space:

```
1 import matplotlib.pyplot as plt
2 import umap
3 import numpy as np
4 from qdrant_client import QdrantClient
5
6 client = QdrantClient(url="https://<cluster-endpoint>:6333", api_key="<
    API_KEY>")
7
8 # Scroll vectors from collection
9 records, _ = client.scroll(
10     collection_name="ecommerce_products",
11     limit=500,
```

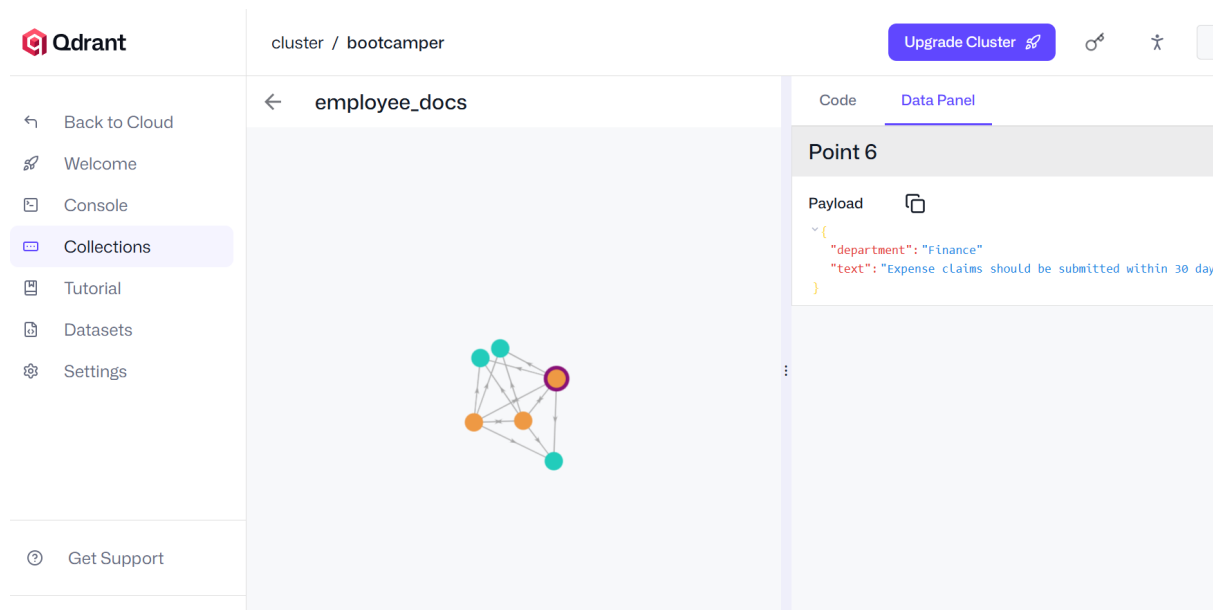



Figure 4: Data Points, and Cosine Score

```

12     with_vectors=True,
13     with_payload=True
14 )
15
16 vectors = np.array([r.vector for r in records])
17 labels = [r.payload.get('category', 'unknown') for r in records]
18
19 # Reduce dimensions using UMAP
20 reducer = umap.UMAP(n_components=2, random_state=42)
21 embedding_2d = reducer.fit_transform(vectors)
22
23 # Plot Graph
24 plt.figure(figsize=(10, 7))
25 plt.scatter(embedding_2d[:, 0], embedding_2d[:, 1], alpha=0.7, c='blue'
26 )
27 plt.title("2D Projection of Vector Topology (HNSW Collection)")
28 plt.xlabel("UMAP Dimension 1")
29 plt.ylabel("UMAP Dimension 2")
30 plt.grid(True)
31 plt.savefig("vector_graph.png")
32 plt.show()

```

Listing 5: Generate 2D Projection Graph of Qdrant Vectors

7 Data Update and Retrieval Strategies (Python Code Snippets)

7.1 Strategy 1: Vector & Payload Upsert (Insert or Update)

Upserting updates existing points if the ID matches or creates a new point if it does not exist.

```

1 from qdrant_client import QdrantClient
2 from qdrant_client.models import PointStruct

```

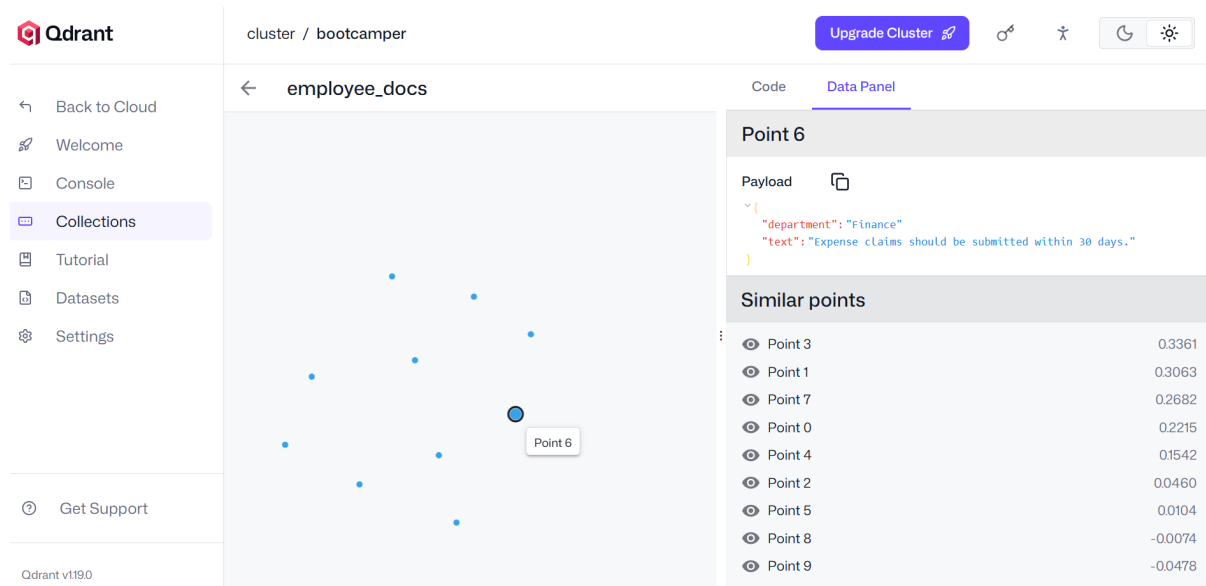


Figure 5: Data Points, and Cosine Score

```

3
4 client = QdrantClient(url="https://<cluster-endpoint>:6333", api_key="<
  API_KEY>")
5
6 # Update Payload for an existing point (ID: 101)
7 client.set_payload(
8     collection_name="ecommerce_products",
9     payload={"price": 29.99, "in_stock": False},
10    points=[101]
11 )
  
```

Listing 6: Upsert and Update Vector/Payload

7.2 Strategy 2: Similarity Top-K Vector Search

Find the top- K nearest items based on cosine similarity vector search.

```

1 from qdrant_client import QdrantClient
2 from sentence_transformers import SentenceTransformer
3
4 client = QdrantClient(url="https://<cluster-endpoint>:6333", api_key="<
  API_KEY>")
5 encoder = SentenceTransformer('all-MiniLM-L6-v2')
6
7 query_text = "wireless noise cancelling headphones"
8 query_vector = encoder.encode(query_text).tolist()
9
10 search_results = client.search(
11     collection_name="ecommerce_products",
12     query_vector=query_vector,
13     limit=5 # Top-K
14 )
15
16 for result in search_results:
17     print(f"ID: {result.id} | Score: {result.score:.4f} | Payload: {
      result.payload}")
  
```

Listing 7: Top-K Similarity Search

7.3 Strategy 3: Filtered Top-K Similarity Search (Hybrid Filtering)

Combine semantic vector search with boolean payload filters (e.g., price range, category).

```

1 from qdrant_client import QdrantClient
2 from qdrant_client.models import Filter, FieldCondition, Range,
  MatchValue
3 from sentence_transformers import SentenceTransformer
4
5 client = QdrantClient(url="https://<cluster-endpoint>:6333", api_key="<
  API_KEY>")
6 encoder = SentenceTransformer('all-MiniLM-L6-v2')
7
8 query_vector = encoder.encode("gaming laptop").tolist()
9
10 # Define strict payload filter
11 search_filter = Filter(
12     must=[
13         FieldCondition(
14             key="category",
15             match=MatchValue(value="Electronics")
16         ),
17         FieldCondition(
18             key="price",
19             range=Range(lte=1500.00, gte=500.00)
20         )
21     ]
22 )
23
24 filtered_results = client.search(
25     collection_name="ecommerce_products",
26     query_vector=query_vector,
27     query_filter=search_filter,
28     limit=3
29 )
30
31 for res in filtered_results:
32     print(f"Match: {res.payload['name']} | Price: {res.payload['price']} | Score: {res.score:.4f}")

```

Listing 8: Filtered Vector Search

8 Summary

You now have a fully operational Qdrant Cloud instance running on GCP Free Tier. You can manage collections visually using the Web UI, upload batch datasets from CSV and JSON files, monitor HNSW indexing stats, and programmatically query/update vectors using advanced similarity filtering strategies in Python.